PYTHON (IMPORTANT QUESTIONS)

Unit 1: Fundamentals of Python

Theoretical Questions

1. What are keywords and identifiers in Python? Provide examples.
2. Explain the different data types in Python with examples.
3. Describe Python blocks and their significance.
4. Differentiate between `pass`, `continue`, and `break` in Python loops.

Programming Questions

1. Write a Python program to check if a number is even or odd.
2. Write a Python program to demonstrate list comprehensions by squaring numbers from 1 to 10.
3. Create a dictionary of students' marks and print only the names of students scoring above 80.
4. Using a `for` loop, iterate over a dictionary and print its keys and values.
5. Write a program to remove vowels from a user-input string.
6. Write a program to create a list of fruits, add a new fruit, sort the list, and delete the last fruit.
7. Demonstrate the use of a `for` loop with ranges, strings, and dictionaries.
8. Use comprehensions to create a list of squares for numbers from 1 to 10.

Unit 2: Functions, Modules & Packages

Theoretical Questions

1. What is a lambda function? Write its syntax and an example.
2. Explain the difference between a module and a package in Python.
3. What is exception handling? Why is it important?
4. Discuss Python's built-in modules like `math` and `datetime` with examples.

Programming Questions

1. Write a Python function to check if a number is prime.
2. Create a custom module named `calculator` with functions for addition, subtraction, multiplication, and division. Import it and use its functions in another program.
3. Write a Python program to handle a `FileNotFoundError` while opening a file.
4. Implement a decorator function to log the execution time of another function.
5. Write a program to calculate the factorial of a number using recursion.

- Implement a function to check if a number is even or odd and handle exceptions for non-integer input.

- Create a module called `math_utils` with functions for addition, subtraction, multiplication, and division. Import and use it in another script.
- Write a program to create a custom exception class to validate age for voting eligibility (age >= 18).

Unit 3: Python Object-Oriented Programming

Theoretical Questions

1. Define class and object with real-world examples.
2. Explain the concept of inheritance in Python with an example of single and multiple inheritance.
3. What is operator overloading? Provide an example.
4. Discuss the significance of the `__init__` method in Python classes.

Programming Questions

1. Create a Python class `Employee` with attributes like `name`, `id`, and `department`. Add methods to display employee details.
2. Write a Python program to demonstrate method overriding in inheritance.
3. Implement multithreading where one thread prints squares of numbers and another prints cubes.
4. Write a program to validate email addresses using regular expressions.
5. Create a class `Library` with attributes like `Acc-Number`, `Title`, and `Author`. Implement methods to compute fines and display book details.

1. Write a program to demonstrate method overriding in inheritance.
2. Implement a program to check password validity using regular expressions (conditions: length, special characters, numbers, etc.).
3. Write a multithreading program where one thread calculates the square of numbers and another calculates the factorial.

Unit 4: Python Database Interaction Using MongoDB

Theoretical Questions

1. What is NoSQL? Differentiate between SQL and NoSQL databases.
2. Explain the advantages of using MongoDB with Python.
3. What are CRUD operations in MongoDB?
4. Discuss the significance of collections and documents in MongoDB.

Programming Questions

1. Write a Python program to create a MongoDB collection named `Students` with fields like `name`, `roll_no`, and `marks`. Insert data into it.

2. Implement a Python program to fetch all documents from a MongoDB collection.
3. Write a Python program to delete a document with a specific `roll_no` in MongoDB.
4. Perform the following operations using MongoDB:
    - Display all employees in the "Finance" department.
    - Update the salary of an employee with a specific `id`.
    - Write a Python program to create a MongoDB collection `Employee` with fields like `Name`, `ID`, and `Salary`. Insert multiple records.

1. Perform the following MongoDB operations:
    - Retrieve employees with a salary between 5000 and 10000.
    - Update the phone number of an employee with `ID = 123`.
    - Delete an employee with `Name = John`.
2. Write a Python program to display documents from a MongoDB collection in descending order of age.
3. Create a MongoDB collection `Books` and insert records with fields like `Title`, `Author`, and `Price`. Update the price of a specific book.
    -

Unit 5: Web Development Using Django

Theoretical Questions

1. What is Django? Discuss its architecture.
2. Explain the purpose of Django models and views.
3. What are Django templates? How are they used in rendering dynamic content?
4. Discuss Django’s authentication framework.

Programming Questions

1. Create a simple Django project to display a “Hello World” page.
2. Implement a Django model for a library system with fields like `BookTitle`, `Author`, and `ISBN`. Write views to list and add books.
3. Build a Django form to take user input for name and email, and display it on the next page.
4. Create a Django Rest Framework (DRF) API to handle CRUD operations for a `Product` model.
5. Create a Django project to display a “Welcome” message on the homepage.

1. Implement a Django model for a student database with fields like `Name`, `Roll Number`, and `Class`. Add views to create and display records.
2. Build a Django form to accept user input (name and email) and display it on a success page.
3. Develop a Django Rest Framework API to handle CRUD operations for a product inventory system.